



FORMATO PARA ELABORACIÓN DE TALLERES ESTUDIANTILES

Departamento:

CIENCIAS DE LA COMPUTACION

Carrera: ELECTRONICA Y AUTOMATIZACION

Asignatura: FUNDAMENTOS DE PROGRHMACION
NRC: 28561

Apellidos y nombres del estudiante: JHOSUE ANGAMARCA,
FRANCISCO COMINA, DANIEL IZA

ANTECEDENTES

Uso de Arduino y simuladores como Wokwi para desarrollar proyectos electrónicos virtuales.
Aplicación de funciones como tone() para la generación de sonidos mediante buzzer.

OBJETIVO

Programar la melodía “Los Pollitos Dicen” en el simulador Wokwi utilizando el buzzer y la función tone() de Arduino, aplicando estructuras de datos y control.

DESARROLLO

Ejecutar en el simulador

1. Haz clic en el botón (Play) para iniciar la simulación.
2. Escucha la melodía desde el buzzer virtual.
3. Ajusta el tiempo o frecuencia para cambiar el ritmo o la tonalidad.

Experimenta

- Cambia NOTE_C4 por NOTE_C5 para subir una octava.
- Duplica el bloque del for dentro de setup() para repetir la canción.
- Agrega una pausa más larga entre frases usando delay(500).

// Los Pollitos Dicen - versión modificada para Wokwi

```
#define SPEAKER_PIN 8
```

```
#define NOTE_C4 262  
  
#define NOTE_D4 294  
  
#define NOTE_E4 330  
  
#define NOTE_F4 349  
  
#define NOTE_G4 392  
  
#define NOTE_A4 440  
  
#define NOTE_B4 494  
  
#define NOTE_C5 523
```

```
int melodia[] = {  
  
    NOTE_C5, NOTE_D4, NOTE_E4, NOTE_C5,  
  
    NOTE_C5, NOTE_D4, NOTE_E4, NOTE_C5,  
  
    NOTE_E4, NOTE_F4, NOTE_G4,  
  
    NOTE_E4, NOTE_F4, NOTE_G4  
};
```

```
int duraciones[] = {  
  
    4, 4, 4, 4,  
  
    4, 4, 4, 4,  
  
    4, 4, 2,  
  
    4, 4, 2  
};
```

```
void setup() {  
  
    pinMode(SPEAKER_PIN, OUTPUT);
```

```

for (int nota = 0; nota < 14; nota++) {

    int duracionNota = 1000 / duraciones[nota];

    tone(SPEAKER_PIN, melodia[nota], duracionNota);

    int pausa = duracionNota * 1.30;
    delay(pausa);

    noTone(SPEAKER_PIN);
}

delay(500);
for (int nota = 0; nota < 14; nota++) {

    int duracionNota = 1000 / duraciones[nota];

    tone(SPEAKER_PIN, melodia[nota] * 2, duracionNota);

    int pausa = duracionNota * 1.30;
    delay(pausa);

    noTone(SPEAKER_PIN);
}

}

void loop() {

}

```

CONCLUSIONES

Es posible reproducir melodías utilizando Arduino mediante programación estructurada.
La función `tone()` permite generar sonidos, mientras que `noTone()` detiene la reproducción.
El uso de arreglos facilita la organización de notas y tiempos en una canción.

RECOMENDACIONES

Experimentar con diferentes notas y duraciones para crear nuevas melodías.
Documentar el código para mejorar su comprensión y mantenimiento.
Utilizar simuladores como Wokwi antes de implementar en hardware real.

ACTIVIDAD DE REFLEXIÓN

1. ¿Qué función cumple `tone()` y por qué usamos `noTone()` después?

La función `tone()` se utiliza para generar una señal de sonido en un pin específico del Arduino, produciendo una frecuencia determinada que corresponde a una nota musical.

Se usa `noTone()` después para detener ese sonido. Esto es necesario porque, si no se detiene, la nota seguiría sonando indefinidamente o se mezclaría con la siguiente, afectando la calidad de la melodía.

2. ¿Qué sucede si aumentas el valor de 1.30 en la línea `int pausa = duracionNota * 1.30`?

Si aumentas ese valor, el tiempo de pausa entre cada nota será mayor, esto provoca que, la melodía se escuche más lenta, haya mayor separación entre notas y el ritmo general cambie, haciéndose más pausado

3. ¿Qué estructura de control se usa para recorrer la melodía y por qué?

Se utiliza un **bucle for**, se usa porque, permite recorrer automáticamente los arreglos de `melodia[]` y `duraciones[]`. Ejecuta el mismo procedimiento para cada nota y controla el número de repeticiones mediante un contador